# Homework
EVOLVING DOCUMENT
Plaksha: Robot motion simulation and optimization

-Andy Ruina

Date of this version: April 30, 2026

Hyperlinks look like this.

## 1 Homework Standards and Format

The output of your work this semester is a portfolio of problem solutions which you understand. All homework is aimed at that. Each problem solution should be in a folder. The name of the folder should be, for example: "2. Numerical Derivative". Inside the folder should be any relevant Matlab files and a document. The document could be a pdf, Word, Google Docs, LateX w/ pdf, or a scan of hand written work. Anything that is clear. Figures could be separate files that you refer to in the text or could be included in the text file. The general goal is that in 10 years you can open a folder and understand the problem and the solution.

**Solution quality.** You need a brief problem restatement so that a reader knows what you are trying to do. Something like: "Given $f(x)$ find $x^*$ so $f(x^*) = 0$." Just enough. Not all of the details. An ideal solution will convince someone who has no reason to trust you, that your solution is correct. It will be such that someone who doesn't know how to do the problem can learn how from your solution. It will explore the problem from various angles, considering special and extreme cases, general trends etc. You need a brief problem restatement. You need any graphs that illustrate important features of the solution as well as a still shot of any simulation. And you need enough words to be clear.

**Code.** Ideally, your code should be so well commented that anyone who does not know how to write the code, can learn it from your code and comments.

**Lets get real.** If I, Andy, was to do the best job I can do on the problems below — problems which I know how to solve — using the standards above, it most problems would take me many hours. It takes a lot of time to be complete, exploratory, and expository. You do not have time to do a perfect job on all of the problems. Not in this course or any other course. Instead, what is written above is a target you want to vaguely aim for. It is not something that you can do regularly. So that you learn how, now and again you should try actually do a great job for some parts of some problems.

**Grading.** To minimally pass the course, by the end of the semester you should be able to do, on your own, with the only help being MATLAB syntax,

Difficulty = 1 `D = 1` problems. D1 To pass the course.
Difficulty = 2 D2 To get more than passing.
Difficulty = 3 D3 To get a high grade.
Difficulty = 4 D4 To get a top grade.

## 2 Problems

1. **Matlab**. Master all parts of the Pratap Matlab book that we posted on Moodle. D = 1.
2. **Numerical derivative.** Use the definition of derivative to calculate the derivative of $\sin(x)$ at $x = 1$. D = 1
   a) Compare that to the exact value (exact value is $\cos(1)$). [Practically speaking, 'exact' means the full computer precision before the compounding of roundoff errors. That is, it is the roundoff error of numbers and of elementary functions.] D =1.
   b) Plot error vs $h$ (log log plot) and note that there is a best $h$ because of the competition between Method error and roundoff error.
   c) Challenge: Using the Matlab resolution of Matlab's smallest number "eps" (look it up), estimate (by analysis) the best $h$ for this calculation and compare It with the minimum on your graph. D = 3.
3. **Tutorial problem.** Complete the exercises from the Tutorial on Thursday morning 29 January. D = 2.
4. **Euler solver.** D = 1. Demonstrate that you can write code equivalent to the code shared with you on 30 January. That code separates the solution of an ODE into
   i) a script (problem set up) and two matlab functions
   ii) The Right Hand Side (RHS) function; and
   iii) The Euler solver (that can solve any set of first order ODEs). If you do that first, you can use it to make the error plots above. You should be able to write this using Matlab help and no more (no Google, no GPT, no copilot).
5. **Make up a problem.** Cook up, or find, some ODE, or set of first order ODEs, and solve them and explore something hopefully-interesting about the solution. D = 2.

6. **Higher order methods.**

   **a)** D = 1. Calculate the derivative of $\sin(x)$ using central difference. Make a log log plot of error vs h. On one plot show Euler's method and the central difference method. You should notice that

   - the best $h$ is slightly bigger with central difference than Euler
   - for given $h$ the error is much smaller with central difference
   - the slope of the curve for central difference is about 2, twice the slope of the curve for Euler (about 1). This indicates that the error for Euler is proportional to $h$, and for central difference the error is proportional to $h^2$. Hence Central Difference is a so called 'second order' method, whereas with Euler, the slope is one indicating that error is proportional to $h$ so it is called a 'first order' method

   **b)** Write a new Matlab function that solves arbitrary ODEs using 'the midpoint method'. This is only a slight modification of the Euler solver. Test it with the differential equation $\dot{x} = x$ with initial condition $x(0) = 1$, evaluated at $t = 1$. On one log log plot show the error as a function of $h$ for both the Euler method and the midpoint method. D = 2.

7. **Spring and mass (2D).** One end of a negligible-mass spring $(k, L_0)$ is pinned to the origin, the other to a mass $(m)$. There is gravity $(g)$. Initial Conditions (ICs): The initial position is $\vec{\boldsymbol{r}}_0 = x_0\hat{\boldsymbol{\imath}} + y_0\hat{\boldsymbol{\jmath}}$, and the initial velocity is $\vec{\boldsymbol{v}}_0 = v_{x0}\hat{\boldsymbol{\imath}} + v_{y0}\hat{\boldsymbol{\jmath}}$. Motion starts at $t = 0$ and ends at $t_{\text{end}}$.

   **a)** Find the Equations of Motion (EoM). D = 1.

   **b)** Assume all parameters and IC's above are given. D = 1.

   **i)** Plot the trajectory of the mass.

   **ii)** Animate the trajectory of the mass.

   **c)** How many ways can you think of checking the numerical solution? Find as many as you can, and do the check. D = 1 (find one) to D = 4 (find many). A list of things to check is started here:

   **i)** $k = 0$, all else arbitrary: The motion is parabolic flight (including falling straight down as a special case) [Why? The system is then just ballistics from freshman physics];

   **ii)** $x_0 = 0, v_{x0} = 0$, all else arbitrary: The motion stays on the y axis [Why? There is no force in the $x$ direction if the mass is on the $y$ axis. Because the initial velocity has no $x$ component, the mass never leaves the $y$ axis;

   **iii)** $g = 0, \vec{\boldsymbol{v}}_0 = \vec{\boldsymbol{0}}$, all else is arbitrary: The motion stays on a radial line. And, if the motion does not cross the origin, the motion is that of a harmonic oscillator (sinusoidal oscillations, check by plotting, say $x$ vt $t$. [Why? (Advanced) Write the EoM and EoMs in polar coordinates

   $\Rightarrow \quad m\ddot{r} = -k(r - L_0)$

   $\Rightarrow \quad$ the harmonic oscillator equation, $m\ddot{r}^* = -kr^*$, where $r^* \equiv r - L_0$.

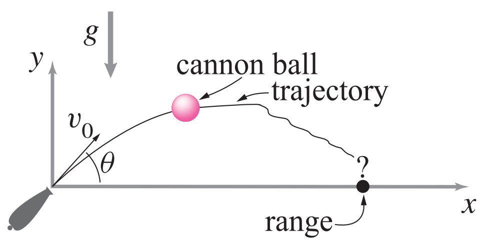
   **iv)** $L_0 = 0$, all else is arbitrary: ____?____. [Why? ____?____.] The question marks indicate the interesting features of the solution and the explanation for those interesting features.

   **v)** *etc.*

   **vi)** *etc.*

**vii)** ...

8. **Cannon ball.** A cannon ball $m$ is launched at angle $\theta$ and speed $v_0$. It is acted on by gravity $g$ and a viscous drag with magnitude $|cv|$.
   - **a)** Find position vs time analytically (optional, for those who like theory) D = 3.
   - **b)** Find a numerical solution using $\theta = \pi/4$, $v_0 = 1\,\mathrm{m/s}$, $g = 1\,\mathrm{m/s^2}$, $m = 1\,\mathrm{kg}$, $c = 1\,\mathrm{kg/s}$. D = 1.
   - **c)** Compare the numeric and analytic solutions. At $t = 2$ how big is the error? How does the error depend on specified tolerances or step sizes? D = 3.
   - **d)** Use larger and larger values of $v_0$ and for each trajectory choose a time interval so the canon at least gets back to the ground. Plot the trajectories (using equal scale for the $x$ and $y$ axis. Plot all curves on one plot. As $v \to \infty$ what is the eventual shape? [Hint: the answer is simple and interesting.] D = 2.
   - **e)** For any given $v_0$ there is a best launch angle $\theta^*$ for maximizing the range. As $v_0 \to \infty$ to what angle does $\theta^*$ tend? D = 3. Justify your answer as best you can with careful numerics, analytical work, or both. D = 4.




9. **Statics vs Dynamics.** 2D. Equilibrium and dynamics of a point mass $m$ at $\vec{\boldsymbol{r}}_C$. Gravity $-g\hat{\boldsymbol{\jmath}}$ points down. Points A and B are anchored at $\vec{\boldsymbol{r}}_A$ and $\vec{\boldsymbol{r}}_B$. Springs and parallel dashpots connect the mass to A and B. The spring constants are $k_A$ and $k_B$. Their rest lengths are $L_A$ and $L_B$. Parallel to the springs are dashpots $c_A$ and $c_B$. The mass also has a drag force proportional to speed through the air $c_d$.
   The dynamic state of the system is described with $z = [x, y, v_x, v_y]'$ and the static state by $z = [x, y, 0, 0]'$.
   - **a) Basic Statics.** Given parameters and an initial guess, find an equilibrium position of this system. If you have an example you like, post it and people can compare solutions. D = 2.
   - **b) Basic Dynamics.** Given parameters and an initial condition, find the motion. D = 1.
   - **c) All equilibria.** Given parameters, attempt to find all equilibria. How many are there? By varying parameters, what is the most and least number of equilibrium points there can be? D = 3.
   - **d) Stability of equilibrium.** Given an equilibrium point, find it's stability by using a dynamic simulation that starts near the equilibrium; (this part deleted and moved to a later problem.).

   Make appropriate plots, animations and comparisons.

10. **Linear ODEs.** Although we are emphasizing numerical solution of arbitrarily complex ordinary differential equations, there are some simple equations you should know the solution to. Most of these are summarized on pages 198 - 201 of the Ruina/Pratap book. The method of solution that I recommend is to guess, then constrain your guess by plugging it into the ODE and finding any free constants. Find the general solution

to these ODEs. You should *know* these, any time and any place, for the rest of your technical life. The first problem below shows the idea. So does one of the tutorials you have done.

**a)** What is the general solution to the ODE $\dot{x} = 7x$?
**Solution.** Guess: $x = Ce^{\lambda t}$ (where $C$ and $\lambda$ are as yet undetermined constants). Plug guess into the ODE:

$$
\begin{aligned}
d/dt(Ce^{\lambda t}) &\stackrel{?}{=} 7Ce^{\lambda t} \\
\Rightarrow \quad C\lambda e^{\lambda t} &\stackrel{?}{=} 7Ce^{\lambda t} \\
\Rightarrow \quad \lambda &= 7
\end{aligned}
$$

This implies that $x = Ce^{\lambda t}$ solves the ODE $\dot{x} = 7x$ for $\lambda = 7$ and for *any* value of C. D = 1.

**b)** Find a solution for $\dot{x} = -3x$. Find the general solution. Use that to solve the Initial Value Problem (IVP): x(0) = 5. Compare the solution with ode45 and plot both on the same plot (one of them should be a dotted line because they may be so similar that it would/will look like a single solution). D = 2.

**c)** Consider the ODE: $\ddot{x} = 9x$. Note, for this problem and the remaining problems: For these linear homogeneous equations, any linear combination of two solutions is also a solution. Find the general solution. Use guessing, don't try to derive the solution. How to guess? Think of a function, put it in the equation, and see if it 'satisfies' the equation. If not, try another. You should find two independent solutions (one is not a multiple of the other). D = 1. Note, that you can also write the general solution using $\cosh(t)$ and $\sinh(t)$ (Why?). D = 2.

**d)** Same as above for $\ddot{x} = -4x$. Note, unlike the case with the first order ODE, the sign change changes the whole structure of the solution. D = 1.

**e)** Nowhere in this and the following subproblems should there be any use of determinants or polynomials. Only guessing or other methods. Find a solution to $\dot{z} = Az$, where $z$ is a two element vector and $A$ is a given $2 \times 2$ matrix of constants.
[Hint: guess that $z = e^{\lambda t} \cdot v$, where $v$ is a yet to be determined two element constant vector.]. At this point leave this abstract. D = 1.

**f)** Still abstract, find the general solution. D = 1.

**g)** Still abstract, solve the initial value problem $z(t = 0) = z_0$ where $z_0$ is a given column vector of two numbers. D = 1. D = 2.

**h)** D = 2. Repeat the whole sequence above with $A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$ and $z_0 = \begin{bmatrix} 5 \\ 0 \end{bmatrix}$.

**i)** Compare the above solution with ode45, either with time plots or with phase plane trajectories, plotting the exact solution on top of the numerical solution, making one of the two with a dotted line. D = 2.

**j)** Consider this IVP:

$$\dot{z} = Az \qquad \text{with} \qquad z(0) = z_0.$$

Make up a random $4 \times 4$ matrix $A$. Make up a random initial condition $z_0$. Solve the IVP two ways: 1) with eigen-values and eigenvectors (using the MATLAB command `eig`) and exponential functions; and with `ode45`. Compare the solutions on the same plot (plot whatever you like), making one of the curves dotted. D = 3. Don't use the matrix exponential for any part of this. D = 3.

**11. Linear stability of springs and mass.** Back to the situation of problem **9.**.

**a)** Pick parameters of your choice (with at least one of the two springs having non-zero rest length or at least one dashpot being non-zero). D = 1.

**b)** Mark both anchor points. Draw lines from the anchors to the mass representing the springs. Make the mass a clear bright disk. Draw thin-line circles around both anchor points representing the rest lengths of the springs (D = 1). Mark all fixed points that you find (D = 2). Draw the path of the mass from the start up to the present time (D = 1). Make the animation as big, smooth and clear as you can (D = 2). Gloat with pride and show a friend or relative (D = 4).

**c)** Try various initial conditions and get a sense of where the equilibrium points are in relation to the spring-rest-length) circles, and how the motion behaves in relation to the anchor points and the equilibrium points (D = 2 - 3).

**d)** Bonus. Sometimes the there are equilibrium points *very* close to anchor points. Can you explain why? D = 4.

**e)** Using the Jacobian supplied by `fsolve` at one of the stable fixed points, solve the linearized equations for motion for an initial condition near the fixed point. D = 3.

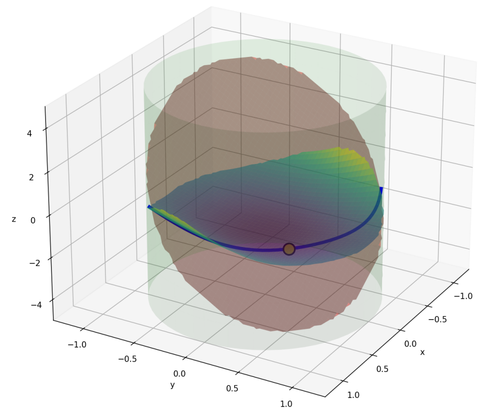
**f)** Using the same initial condition, use ODE45 to find the motion. D = 2.

**g)** Find an unstable fixed point. Repeat the exercise above using a long enough time to see some complexity in the motion. D = 3.

**h)** Setting both spring rest-lengths to zero and both dashpots to zero, linearize about the stable fixed point. Solve the same IVP for some initial condition far from the fixed point (and with non zero initial velocity). Compare with the solution of the same IVP using ODE45. Plot the two solutions on the same plot, with one curve dotted. D = 3 - 4.

**i)** Bonus. If both springs have zero rest length, and the two dashpots are zero, the whole system turns out to be linear. Why? D = 3 - 4.

**12.** Numerical optimization. Consider the surface $z = (x(x-y))^2 - exp(-x^2-y^2) - \cos(x)$ Consider the plane $x + y + z/3 = 0$. Find the minimum value of $z$ that is on the surface, on the plane and inside the unit circle. Find the $x$ and $y$(called 'argmin'). Do this with a grid search (a bit tricky so will take some thought). D = 3. Then learn fmincon and try again ( D= 2). Start with easier problems for practice. Make a 3D plot showing the surface, the plane and the minimum point (D = 3).
According to Claude the answer is $(x, y, z) \approx (0.236, 0.364, -1.800)$




**13. Periodic Solutions.** Finding periodic motions. In all cases, try to be clear in your mind, and in what you show, what things are due to numerical approximation and what are real phenomena. For example, If something is actually periodic, but appears as only approximately periodic, it will become closer and closer to periodic if you increase your integration accuracy.
Note: Unless you know a shortcut, the only way to find a periodic solution is by root finding of some kind.
The actual homework problem here, is the last item. What is above that is necessary preparation. Do the problems in order.

**a)** Verify, using numerical solutions, that for $\ddot{\vec{\boldsymbol{r}}} = -(k/m)\vec{\boldsymbol{r}}$ that for any and all initial conditions you get periodic motions (straight lines, circles, or elipses). D = 2. Extra credit, verify this with pencil and paper (D = 3). This is a 'linear' central force'.

**b)** Consider $\ddot{\vec{\boldsymbol{r}}} = -G(M/r^3)\vec{\boldsymbol{r}}$ (this is an inverse square central gravitational force towards a fixed mass $M$. The mass $m_1$ of the moving mass drops out because the gravitational force is proportional to mass *This looks like inverse cube, why call it 'inverse square'?*). Verify with numerical solutions that for any and all initial conditions, either: i) the solution goes off to $\infty$, or ii) The solution is periodic and is either a circle, a straight line, or an elipse. D = 2 - 3.

**c)** Consider $\ddot{\vec{\boldsymbol{r}}} = -kr^{n-1}\vec{\boldsymbol{r}}$. Pick any $n$ that you like, and any $k > 0$ that you like, but not $n = 1$ nor $n = -2$ (the previous two cases). Show by numerical experiments that a) if you start from rest you get straight line motion towards the origin (D = 1); and b) if you start with $[x, y, v_x, v_y] = [1, 0, 0, v_y$ and you pick $\vec{\boldsymbol{y}}$ appropriately you get circular motion (D = 2).

**d)** For your given $k$ and $n$ ($\neq 1$ or -2) show that for random initial conditions you generally do not get periodic motion (you have to integrate for a long time. Read the directions for this problem at the top. D = 2. Note: periodic is not the same as 'quasi-periodic' (look it up).

**e)** For this central force problem, using $n > -3$, find a periodic motion that is not a straight line and not a circle (using root finding of some kind). D = 3. ( For $n \leq -3$ what goes wrong?. D = 4.).

**f)** Using the 2-mass, 2-spring problem we have been toying with in previous weeks, with all damping = 0, and with at least one spring having non-zero rest length, with no-zero gravity, and with the anchor points not vertically aligned, find a periodic motion. D = 3. These given details matter.

**14. Stability of periodic motion.** Don't do this probem unti you are comfortable with all of the problems above. For any parameters you like in the problem above, possibly different parameters for each part, find

**a)** A solution any normal person would call chaotic. D = 2-3.

**b)** An unstable periodic motion D = 3.

**c)** A neutrally stable periodic motion D = 3.

**d)** An asymptotically stable periodic motion D = 4. The 2-spring mass system above will not work for this (well, only in a very special case). Best to find a different system. D = 4.

**e)** Compare the stability from simulations with the semi-analytic stability calculation (D = 4):

> Evaluate the stability using the eigenvalues of the numerically-found Jacobian of the return map at the fixed point. One eigenvalue is always 1 (Why? D = 4). If all but one eigenvalue has magnitude ¡1 the periodic motion is stable. If more than one eigenvalue has magnitude of 1 and the rest have magnitude ¡1, the periodic motion is neutrally stable. If any eigenvalue is greater than one in magnitude, the solution is unstable. Compare this semi-analytic calculation of stability with the numerically found motons starting near the fixed point.

**15. Initial Value Problem (IVP) with collocation.**

**Objective** This is the first of several problems on the general topic of trajectory optimization. Here are the topics:

- Initial Value Problems (IVP), review
- Boundary Value Problems (BVP) via shooting
- Collocation Methods

The first few problems are warm up. Note, these problems all have short-cuts or analytic solutions. So, the methods asked of you are not needed. Harder problems don't have such shortcuts, and the methods then *are* needed. So, take advantage of the easy problems to learn the methods.

**Euler method solution of ODE** (review).
Consider

$$\ddot{x}(t) = -1, \quad t \in [0, 1] \tag{1}$$

**a)** Convert this to a first-order system (D= 1)

$$\dot{x} = v, \quad \dot{v} = -1, \quad x(0) = 0, \quad v(0) = 0.5$$

**b)** Implement forward Euler integration (This is review, you've done it.). The Euler integration formula, which you should know inside and out by now, is $z_{n+1} = z_n + h\dot{z}_n$. D = 1.

**c)** Implement Trapezoid Integration. The formula here is that $z_{n+1} = z_n + h(\dot{z}_n + \dot{z}_{n+1})/2$. D = 2. This problem has a little bit of a trick to it. Because $a$ is constant, you can use the trapezoid rule to find $v_n$. Then you can use that to find $x_n$. For most ODEs, the trapezoidal rule is an implicit method and depends on root finding (for example, the collocation methods below).

**d)** Simulate trajectory for different step-sizes (.1 and .001$s$) using the two integration methods implemented above and observe the differences in state trajectory. What do you observe, and why? (You should ace this, without much thought. These are essentially the same problems that you did at the start of the semester.) D = 2.

**16. Boundary Value Problem via Shooting.** You can do this problem with pencil and paper. Don't do that until you are asked. Find $\dot{x}(0) = v_0 = s$ to solve this problem

$$\ddot{x}(t) = -1, \tag{2}$$
$$x(0) = 0, \quad x(1) = 0 \tag{3}$$

Solve this with $h = 0.25$. Then do it with $h = 0.1$. How?

**a)** Assume a value for $s$. Define the shooting function:

$$F(s) = x(1; s)$$

In words, $x(1; s)$ is $x$ at $t = 1$ if you solve the given ODE using the initial conditions $x(0) = 0$ and $\dot{x}(0) = s$.

**b)** Plot $F(s)$ vs $s$. D = 1.

**c)** Use a home made (not `fsolve`) root-finding method (bisection/Newton) to find $s^*$ such that (D = 3):

$$F(s^*) = 0$$

**d)** Verify your numerical result with an analytical solution. D = 3.

**17. Collocation (Euler Discretization)** Consider the differential equations: $\dot{x} = v$ and $\dot{v} = -1$. Again we want to solve these equations with the boundary conditions: $x(0) = 0$ and $x(1) = 0$. Euler's method is to solve the following difference equations iteratively, starting with $k = 1$ then $k = 2$ etc.:

$$x_{k+1} = x_k + hv_k \tag{4}$$
$$v_{k+1} = v_k - h \tag{5}$$

But you don't know $v_0 = v(0)$, so you have to guess. Then keep guessing until you find the value of $v_0$ that gets $x$ back to 0 at $t = 1$. That's 'shooting'. We are now going to solve the same Euler-method difference equations, a different way that is not shooting. It's collocation. Pick a value for $N$. To start, use $N = 4$ so you can work this out by hand.

**a)** Formulate the linear system (D = 2):

$$Az = b$$

where:

$$z = [x_1, \ldots, x_N, v_1, \ldots, v_N]'$$

Note that the first $x$ at $t = 0$ is $x_1$. That is $x_1 = x(t_1) = x(0)$ where $t_1 = 0$. The notation is ambiguous. If you think of the argument or subscript as continuous time, then 0 means $t = 0$. If you think of the index or subscript as an index of discrete numbers, than the first one, corresponding to $t = 0$ is associated with the number 1. Why the confusion? Because indexing in Matlab and in math (but not in C or Python) starts at 1. There are $2(N-1)$ such equations (6, if $N = 4$).

**b)** Also enforce the 2 end conditions,

$$x_1 = 0, \quad x_N = 0.$$

Altogether that's 8 linear equations for 8 unknowns. So, $A$ is $8 \times 8$ and $b$ is $8 \times 1$.

**c)** Solve the system and obtain $x(t), v(t)$. (The solution is a set of 4 numbers for $x$ and 4 numbers for $v$.) Use the backslash command (`z = A\ b`) and then separate out $z$ into its parts (`z = [x;v]`). D = 3.

**d)** Compare your solution with the shooting solution in the previous problem where you used $h = .25$. D = 3.

**e)** Now do this again, with $N = 10$. Use loops to populate the elements of the matrix $A$. Compare your solution with $h = .1$ by the shooting method. D = 3.

**18. Optimal Control.** Finally, an optimal control problem. You have a mass $m = 1$ that you want to move from being at rest $x = 0$ at the start, to being at rest at the end $x = 1$ after $T = 1$s. You push it with a force $F$ which you control. $F$ could be any function of time. There are lots of solutions to this problem. But, you want to save energy. Assume that the rate of energy expenditure is $F^2$ so that the total energy is $\int_0^1 F(t)^2\,dt$. The convention in the 'optimal control' community is to call the thing you control, $F$ in this case, $u$. So the problem is, in math language: Now solve:

$$\min_{u(t)} \int_0^1 u(t)^2\,dt \tag{6}$$

This says, find the function $u(t)$ that causes the 'objective function' $J \equiv \int_0^1 u(t)^2\,dt$ to be as small as possible. Well, that would be $u(t) \equiv 0$. But we also need to achieve our task and satisfy the governing equations and get to the right places at the right times. So, this minimization is subject to:

$$\ddot{x} = u, \quad x(0) = 0, \quad v(0) = 0, \quad x(1) = 1, \quad v(1) = 0.$$

**a)** Discretize and solve using `fmincon`. First do this with a small number of discretization points and set this up by hand (like, $N = 4$). Then automate the generation of equations using loops. D =3.

Basically, the hard work is writing the approximations of the objective $J$ and the differential equations, and the boundary conditions, in terms of all as functions involving the unknown values of the $x$'s, $v$'s, and $u$'s. You have a choice between writing the differential equations as

i) Linear constraints, using a matrix $A$ and column vector $b$, as per the methods above, or as

ii) Non-linear constraints.

The first is harder for you, but easier for the computer to solve. The second is easier for you, but harder for the computer to solve. Usually, you don't have the luck of being able to write the differential-equation constraint as linear equations, so it is not a big loss if you don't develop the skill of populating the matrix $A$.

**b)** Plot $x(t)$, $v(t)$, $u(t)$. D = 3.

**c)** Admire the beautiful solution if you use a large $N$ (a large number of collocation points). D = 4.

**19. Trapezoidal discretization** For the optimal control problem above, replace Euler discretization with trapezoidal discretization.

$$x_{k+1} = x_k + \frac{h}{2}(v_k + v_{k+1}) \tag{7}$$

**a)** Implement the scheme. D = 2-3.

**b)** Compare the accuracy with the Euler solution. D = 2-3. How can you estimate the accuracy? D = 3-4.

**20. Mystery Matrix**. Don't look this up until you have solved it, or thought about it for at least one hour. This is not a hard problem.

I have written a Matlab function and given to you, but you are not allowed to look inside of it.

```
function  u = andymatrix(v)
  % A is nxm.   You are given m.
  A  = [...stuff you cannot see...];
  u = A * v;
end
```

**a)** You write a function that finds `A`:  `A = mydetective(m)`. D = 2.

**b)** When could we use this method in this course? (We had a Matlab shortcut, so didn't have to.) D = 3.

**c)** Harder: For *calculating* stability of periodic orbits (not just observing through simulation), we had to use this method. Can you explain that situation and write a function to execute the method. D = 4.

**21. Problem 1 in this set: Optimal Planning and Control**

**Problem Setup (for next several problems.)** Consider a 2D point-mass model (drone/lander):

$$\dot{x} = v_x, \tag{8}$$
$$\dot{z} = v_z, \tag{9}$$
$$\dot{v}_x = u_x, \tag{10}$$
$$\dot{v}_z = u_z - g \tag{11}$$

where $g = 1.62$ m/s$^2$.

**Initial condition:**

$$x(0) = x_0, \quad z(0) = z_0, \quad v_x(0) = v_{x0}, \quad v_z(0) = v_{z0}$$

**Final condition:**

$$x(T) = 0, \quad z(T) = 0, \quad v_x(T) = 0, \quad v_z(T) = 0$$

**Control constraint:**

$$\sqrt{u_x^2 + u_z^2} \leq u_{\max}$$

**State constraint:**

$$z \geq 0$$

**Problem 1: Generate Minimum Control Effort Optimal Plan**

**Objective:**

$$\min_{x(t),,u(t)} \int_0^T ||u(t)||^2 dt$$

**Tasks:**

**a)** Solve the problem using:

- Euler OR trapezoidal collocation with fmincon (or equivalent) with $T = 5s$ and initial condition $x_0 = 5, z_0 = 10, v_{x0} = -1$, $v_{z0} = -2$.

**b)** Plot:

- Trajectory ($x$ vs $z$)
- Control inputs ($u_x$, $u_z$ vs time)
- Thrust magnitude vs time

**Exploratory Subproblems:**

**i) Controllable Region Estimation** Empirically determine the set of initial positions $(x_0, z_0)$ for which the trajectory optimization problem is feasible.

- Fix initial velocities $(v_{x0}, v_{z0})$ and all problem parameters.
- Sweep:

$$x_0 \in [x_{\min}, x_{\max}], \quad z_0 \in [z_{\min}, z_{\max}]$$

  over a grid.
- For each $(x_0, z_0)$:

- Solve the trajectory optimization problem
- Record whether the solver converges to a feasible solution

- Plot a 2D scatter plot:
  - feasible points (successful convergence)
  - infeasible points (failure to converge)

**ii)** **Feasibility Recovery via Parameter Variation** Select one initial condition $(x_0, z_0)$ for which the problem is infeasible. Investigate how feasibility can be restored by modifying problem parameters. Consider:

- Increasing maximum thrust $u_{\max}$
- Increasing final time $T$

For each modification, resolve the optimization problem. Identify which changes lead to a feasible solution. Provide a brief explanation of the observed behavior

**22. Problem 2 in this set: Open-Loop Execution**

Using the optimal control from Problem 1:

**Tasks:**

**a)** Simulate the system forward using the computed control.

**b)** Introduce disturbance:

$$u_{\text{applied}} = \alpha\, u^*, \quad \alpha \in [0.7, 0.9]$$

**c)** Plot:

- Planned vs actual trajectory
- Time vs states

**d)** Compute final error:

$$\|x_{\text{final}} - x_{\text{target}}\|$$

**23. Problem 3 in this set: MPC Tracking (Feedback Control)**

Use the trajectory from Problem 1 in this set as reference.

**At each time step, solve:**

$$\min \sum_{i=1}^{N_p} (x - x^{ref})^\top Q (x - x^{ref}) + u^\top R u$$

subject to the dynamics and constraints described in the problem setup. Use a step size of 100 ms for discretization. $N_p$ here refers to horizon length. So, if planning for 1 sec, with 10 ms of step size, horizon length becomes 10.

**Tasks:**

**a)** Implement MPC with horizon $N_p$.

**b)** At each step:

- Solve optimization

- Apply only first control

**c)** Use the same disturbance as Problem 2.

**d)** Plot:

- Reference vs MPC trajectory
- Control comparison (optimal vs MPC)

**Exploratory Subproblem: Effect of Horizon Length**

**e)** Repeat the MPC implementation for different prediction horizons:

$$N_p \in \{5, 10, 20, 30\}$$

**f)** For each $N_p$:

- Run the closed-loop simulation under the same disturbance
- Record tracking performance

**g)** Plot:

- Trajectory comparison for different $N_p$
- Tracking error:

$$\|x(t) - x^{ref}(t)\|$$

**h)** Compare:

- Final tracking error
- Transient behavior
- Control smoothness

24. **Problem 4 in this set: Time-Optimal Control.** AKA minimum time control
**Objective:** minimize $T$.

$$\min T$$

**Tasks:**

**a)** Formulate the problem with $T$ as a decision variable.

**b)** Use trapezoidal collocation:

**c)** Solve using nonlinear optimization.

**d)** Plot:

- Trajectory
- Control inputs
- Thrust magnitude

**e)** Compare with minimum-energy solution.

**Questions:**

**a)** Is the control saturated? Why?

**b)** How does the trajectory differ from energy-optimal case?

**c)** How does final time compare?

### Conceptual Questions

**d)** Why does open-loop control fail under disturbance?

**e)** Why does MPC succeed?

**f)** What is the key difference between trajectory optimization and MPC?

**25. Optimization discovers walking and running.**

**a)** Before starting this, take some time understanding the problem.

   **i)** **Andy's class** lecture on Thursday 30 April 2026.

   **ii)** **Computer optimization discovers Walking** paper in Nature Magazine. Mostly easy to read. 4 pages in the journal.

   **iii)** **Manoj Srinivasan Lecture 1** on energetics and locomotion. 1 hr and 6 min. (Recommended)

   **iv)** **Manoj Srinivasan lecture 2** about optimization and locomotion. 1 hr 40 minutes (Recommended, specifically about trajectory optimization).

   **v)** Short **News article** about the paper.

   **vi)** **Two minute video** about the paper.

   **vii)** **Fake podcast** about Manoj's paper (18.25 long). Pretty good. Not all perfect (*e.g.,* skip the RK45 stuff).

**b)** The problem.

   **i)** Spend some time with the resources above. **Required**, at least 4 hours on this problem, either studying above, or working on the problem, or some combination. Five hours if you missed class 30 April. If you find it too difficult, spend the time learning from the resources above.

   **ii)** Equivalent to using dimensionless variables, set $m = 1, L = 1, g = 1, d = 0.5, T = 0.5$ . Find as best you can, the optimal gait using the model in the Manoj paper above. This is within reach, but is relatively difficult in that we have not told you precise prescriptions of some of the calculation steps (so to speak).

   **iii)** Given interest and time, explore other parameters or other related models.